

CPP MODULE 02

Relatório Técnico de Arquitetura e Implementação de Ponto Fixo

Desenvolvedor: Leonardo La Rocque

1. INTRODUÇÃO AO CONCEITO DE PONTO FIXO (FIXED-POINT)

O objetivo central do **CPP Module 02** da 42 Network é o desenvolvimento de uma classe em C++98 capaz de abstrair e gerenciar números em formato de **Ponto Fixo**. Tradicionalmente, as linguagens de programação oferecem tipos nativos de ponto flutuante (como `float` e `double`) baseados na especificação internacional IEEE 754. Contudo, em cenários específicos de engenharia de software e hardware, o ponto fixo apresenta vantagens críticas.

1.1 Ponto Fixo vs. Ponto Flutuante

Ao contrário do ponto flutuante, que aloca dinamicamente os bits disponíveis entre sinal, expoente e mantissa para deslocar a vírgula digital conforme a magnitude do número, o ponto fixo define uma partição estática e imutável de bits.

Neste projeto, a classe `Fixed` encapsula um número inteiro de 32 bits (`int _value`), no qual os **8 bits menos significativos (à direita)** são rigidamente reservados para a precisão fracionária, conforme estipulado pela constante estática `static const int _fBits = 8;`.

1.2 Justificativa de Uso na Engenharia de Sistemas

- **Determinismo Absoluto:** Os números de ponto flutuante podem sofrer sutis variações de arredondamento e precisão dependendo do compilador, do sistema operacional ou da arquitetura de hardware (x86 vs ARM). Em motores de física para jogos multiplayer ou em sistemas financeiros complexos, onde cada bit precisa ser perfeitamente replicável em rede, o ponto fixo garante resultados idênticos bit a bit.
- **Performance em Sistemas Embarcados:** Microcontroladores de baixo custo e arquiteturas embarcadas antigas muitas vezes não possuem uma FPU (Floating Point Unit) em hardware. Operações aritméticas com floats nessas arquiteturas são emuladas via software, tornando-as extremamente lentas. Como a classe de ponto fixo opera inteiramente sobre álgebra de inteiros nativos, ela extrai performance máxima do processador.

2. A MATEMÁTICA DO EPSILON E O BIT MENOS SIGNIFICATIVO

Durante o desenvolvimento do `ex02`, ao realizar um incremento simples na memória crua através de `this->_value++`, o terminal exibe o valor flutuante fracionado **0.00390625**. Esse valor não é arbitrário; trata-se do **Epsilon (ε)** da nossa classe.

Como reservamos exatamente 8 bits para a fração, a menor fração binária que conseguimos ligar na tabela de bits corresponde ao primeiro bit posicionado imediatamente à direita do ponto fixo (o bit de menor peso fracionário). Matematicamente, calculamos o valor decimal desse bit através de potências negativas de base 2:

$$2^{-8} = 1 / 256 = 0.00390625$$

Para fins de analogia pedagógica de fácil defesa diante de um avaliador, esse valor funciona exatamente como o "um centavo" (**0.01**) do nosso sistema monetário. Não existe fração decimal inferior a **0.00390625** que nossa classe consiga armazenar. Portanto, qualquer incremento ou decremento direto no valor bruto (raw bits) alterará o número em passos estritos de **0.00390625**.

3. EVOLUÇÃO ARQUITETURAL DO MÓDULO

3.1 Exercício 00: A Forma Canônica Ortodoxa

O estágio inicial (ex00) foca na pura anatomia e conformidade da classe com as regras estruturais clássicas do C++. Para garantir uma gerência de memória segura e prevenir vazamentos ou comportamentos indefinidos em cópias profundas, a classe implementa rigorosamente a **Forma Canônica Ortodoxa**:

1. **Construtor Padrão**: Inicializa os bits em zero (`_value(0)`).
2. **Construtor de Cópia**: Garante que novas instâncias nasçam clonando corretamente instâncias preexistentes através de atribuição profunda.
3. **Operador de Cópia por Atribuição**: Copia dados de forma segura entre objetos já instanciados na memória, incluindo validações para autoatribuição (`this != &other`).
4. **Destrutor**: Encarregado da liberação limpa de escopo.

3.2 Exercício 01: Conversores e Manipulação de Bits (Bitwise)

O ex01 expande a classe adicionando construtores parametrizados e funções conversoras utilizando os operadores de deslocamento de bits à esquerda (`<<`) e à direita (`>>`):

- **Conversão de Inteiro**: Para converter um inteiro bruto `n` para ponto fixo, deslocamos seus bits 8 posições para a esquerda (`n << 8`). Isso é equivalente a multiplicar o número por 256, empurrando o valor inteiro para as casas binárias corretas.
- **Conversão de Float**: Como o operador `<<` não é aceito por floats nativos, realiza-se a multiplicação de `n * (1 << 8)` (ou seja, `n * 256`). O resultado decimal resultante é encapsulado pela função `roundf()` da biblioteca `<cmath>` para arredondar ao inteiro mais próximo, eliminando truncamentos severos de precisão antes de salvar no container `_value`.

4. EXERCÍCIO 02: SOBRECARGA COMPLETA DE OPERADORES

No ex02, a classe ganha autonomia e passa a se comportar de forma idêntica a um tipo nativo primitivo da linguagem. Para isso, foi implementada a sobrecarga de 20 operadores distintos. A tabela abaixo sintetiza essas regras:

CATEGORIA	OPERADORES SOBRECARGADOS	RETORNO EFETUADO	GARANTIA DE CONSTÂNCIA (CONST)
Comparação	> , < , >= , <= , == , !=	bool	Sim (Apenas lê valores sem modificar o estado)
Aritmética	+ , - , * , /	Fixed (Por valor)	Sim (Gera uma nova instância com a resposta da conta)
Pré-Incremento/ Decremento	++a , --a	Fixed& (Referência)	Não (Altera os bits do objeto atual diretamente)
Pós-Incremento/ Decremento	a++ , a--	Fixed (Por valor)	Não (Modifica o objeto, mas devolve cópia do estado anterior)

4.1 Funções Estáticas de Avaliação (Min e Max)

Foram desenvolvidas as funções membro estáticas `Fixed::min()` e `Fixed::max()`. Por serem estáticas (`static`), elas pertencem ao escopo global da estrutura da classe e operam sem o ponteiro oculto `this`, permitindo sua chamada limpa por meio da sintaxe `Fixed::max(a, b)`.

A implementação otimizada utiliza o operador ternário junto aos próprios operadores de comparação já sobrecarregados, de forma transparente:

```
return ((a > b) ? a : b);
```

A classe disponibiliza essas funções em sobrecarga dupla (aceitando parâmetros normais e também instâncias imutáveis via `const Fixed&`), garantindo cobertura total em fluxos de leitura restrita.

5. A ENGENHARIA DO OPERADOR DE INSERÇÃO (<<)

Um ponto vital para demonstrar proficiência técnica de alto nível em C++ diz respeito ao motivo de o operador de inserção de fluxo de saída (`std::ostream& operator<<`) ser obrigatoriamente definido **fora** da classe `Fixed`.

ARGUMENTAÇÃO CRÍTICA PARA DEFESA COMERCIAL/MOULINETTE

"Se definíssemos o `operator<<` como uma função interna (membro) da classe `Fixed`, a semântica de ordem do C++ exigiria que o nosso objeto `Fixed` ficasse posicionado obrigatoriamente do lado esquerdo do operador durante chamadas. Isso forçaria uma sintaxe contra-intuitiva e bizarra como `objeto_fixed << std::cout;`. Para preservar a sintaxe canônica e correta da linguagem (`std::cout << objeto_fixed;`), o operador precisa ser declarado de maneira global e externa, recebendo o fluxo `ostream` como o primeiro argumento por referência."

Internamente, essa função global atua como um tradutor: ela recebe a referência constante do nosso objeto, extrai sua representação legítima em ponto flutuante por meio do método `toFloat()` (o qual possui o modificador `const` no final para assegurar que ler o dado não violará a imutabilidade do objeto) e direciona esse float limpo para a esteira do fluxo de saída, retornando a referência do próprio fluxo para permitir o encadeamento de impressões na mesma linha.